

FPGA AI Suite 2026.1.1 Installation Guide

Printable guide for Windows 11 and Linux/Ubuntu users

Release focus: FPGA AI Suite 2026.1.1

OpenVINO requirement: OpenVINO Toolkit 2025.4

Recommended beginner install: Docker image

1. What this guide covers

This guide gives a practical installation and verification flow for:

1. Windows 11 systems using Docker Desktop.
2. Linux/Ubuntu systems using Docker Engine.
3. The complete FPGA AI Suite Docker image with Quartus included.
4. Basic verification with dla_compiler.
5. ResNet-50 model conversion, compilation, area/performance estimation, and software emulation.
6. Initial Agilex 5 E-Series Modular Development Kit notes.

The current complete image used in this guide is:

```
alterafpga/fpgaaisuite:2026.1.1-quartus
```

Use this image when you want FPGA AI Suite plus Quartus tools in one environment.

2. Important version support note

For FPGA AI Suite 2026.1.1, the official native Linux operating systems are:

```
Ubuntu 22.04 LTS
Ubuntu 24.04 LTS
Red Hat Enterprise Linux 8.10 / 9.4
```

Ubuntu 26.04 is not listed as an officially supported native installation target. On Ubuntu 26.04, the recommended practical path is Docker.

3. Key concepts

FPGA AI Suite components

Component	Purpose
OpenVINO	Converts and represents AI models in IR format
'xml' model file	OpenVINO model graph/layer structure
'bin' model file	OpenVINO model weights
'dla_compiler'	FPGA AI Suite compiler for area/performance/AOT output
'arch' file	FPGA AI IP architecture description
'aot' file	Ahead-of-time compiled model
Runtime	Runs inference through emulation, JTAG, PCIe, or SoC flow
Quartus	Builds/programs FPGA hardware bitstreams

Quartus vs OpenVINO runtime

Quartus builds/configures FPGA hardware. OpenVINO plus the FPGA AI Suite runtime runs inference. Quartus does not replace OpenVINO runtime.

Part A - Windows 11 installation flow

A1. Choose the correct Docker installer

For standard x86-64 Windows workstations, choose:

```
Docker Desktop for Windows - AMD64
```

Do not choose Windows ARM64 unless the machine has an ARM CPU.

A2. Install Docker Desktop

1. Download Docker Desktop from Docker.
2. Install it with WSL 2 backend enabled if prompted.
3. Restart Windows.
4. Open PowerShell and test:

```
docker --version  
docker run hello-world
```

A3. Pull FPGA AI Suite image

For complete FPGA AI Suite plus Quartus:

```
docker pull alterafpga/fpgaaisuite:2026.1.1-quartus
```

For smaller compiler/emulation-only use:

```
docker pull alterafpga/fpgaaisuite:2026.1.1-standalone
```

A4. Create Windows workspace

```
mkdir C:\fpga-ai-work
```

A5. Start container

Complete image:

```
docker run -it --name fpga-ai-suite-2026 -v C:\fpga-ai-work:/workspace alteraf  
pga/fpgaaisuite:2026.1.1-quartus
```

Restart later:

```
docker start -ai fpga-ai-suite-2026
```

A6. How to start FPGA AI Suite each time on Windows

After Docker Desktop is running, open PowerShell and start the existing container:

```
docker start -ai fpga-ai-suite-2026
```

You are inside FPGA AI Suite when the prompt changes from a Windows prompt like:

```
PS C:\Users\name>
```

to a Linux container prompt like:

```
fpga_ai_suite@container_id:~$
```

Then go to your workspace and reload the local FPGA AI Suite work environment:

```
cd /workspace/quickstart
source ./coredla_work.sh
```

If the workspace has not been created yet, use the workspace setup section later in this guide.

To exit the container:

```
exit
```

Do not create a new container every day unless you intentionally want a clean environment. Use `docker start -ai` for daily work.

A7. Verify inside Docker

Prompt should look like:

```
fpga_ai_suite@container_id:~$
```

Run:

```
python3 -c "import openvino; print(openvino.__version__)"
echo $COREDLA_ROOT
dla_compiler --version
quartus_pgm --version
jtagconfig
```

Expected OpenVINO version begins with:

```
2025.4
```

If no FPGA board is connected, `jtagconfig` may show:

```
No JTAG hardware available
```

That is normal.

Part B - Linux/Ubuntu Docker installation flow

B1. Recommended for Ubuntu 26.04

Ubuntu 26.04 is not officially listed as a native FPGA AI Suite target. Use Docker.

B2. Install Docker Engine

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -  
o /etc/apt/keyrings/docker.gpg  
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

If Docker has a repo for your Ubuntu codename, use it. If not, use the Ubuntu 24.04 noble repo:

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docke  
r.gpg] https://download.docker.com/linux/ubuntu noble stable" | sudo tee /etc/  
apt/sources.list.d/docker.list > /dev/null
```

Install Docker:

```
sudo apt update  
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin  
sudo systemctl enable docker  
sudo systemctl start docker  
sudo usermod -aG docker $USER
```

Log out and log back in. Test:

```
docker run hello-world
```

B3. Pull complete FPGA AI Suite image

```
docker pull alterafpga/fpgaaisuite:2026.1.1-quartus
```

B4. Create workspace

```
mkdir -p ~/fpga-ai-work
```

B5. Run container with USB/JTAG support

Use this command if you may connect an FPGA board later:

```
docker run -it \  
--name fpga-ai-suite-2026 \  
--privileged \  
-v /dev/bus/usb:/dev/bus/usb \  
-v ~/fpga-ai-work:/workspace \  
alterafpga/fpgaaisuite:2026.1.1-quartus
```

Restart later:

```
docker start -ai fpga-ai-suite-2026
```

B6. How to start FPGA AI Suite each time on Linux

Open a terminal and start the existing container:

```
docker start -ai fpga-ai-suite-2026
```

You are inside FPGA AI Suite when the prompt changes from the host prompt, for example:

```
sahilpat@sahilpat:~$
```

to the container prompt, for example:

```
fpga_ai_suite@container_id:~$
```

Reload your working directory environment:

```
cd /workspace/quickstart
source ./coredla_work.sh
```

If you need USB-JTAG access and the original container was not created with USB pass-through, recreate it with `--privileged -v /dev/bus/usb:/dev/bus/usb`.

To exit the container:

```
exit
```

If you need to recreate the container:

```
docker rm fpga-ai-suite-2026
```

B7. Verify inside Docker

```
python3 -c "import openvino; print(openvino.__version__)"
echo $COREDLA_ROOT
dla_compiler --version
quartus_pgm --version
jtagconfig
```

Expected examples:

```
OpenVINO: 2025.4.x
FPGA AI Suite: 2026.1.1
Quartus Programmer: 26.1
```

Part C - Workspace setup

C1. Create FPGA AI Suite working directory

Inside Docker:

```
cd /workspace
mkdir -p quickstart
cd quickstart
source dla_init_local_directory.sh
```

If already initialized:

```
source ./coredla_work.sh
```

Verify:

```
echo $COREDLA_WORK
```

Expected:

```
/workspace/quickstart
```

C2. Fix workspace permission issue if needed

If you see:

```
mkdir: cannot create directory: Permission denied
```

Inside Docker:

```
sudo chown -R fpga_ai_suite:fpga_ai_suite /workspace
```

Then retry workspace creation.

Part D - Verify compiler and architecture files

D1. Check Agilex 5 architecture files

```
ls $COREDLA_ROOT/example_architectures | grep AGX5
```

Common files include:

```
AGX5_Generic.arch  
AGX5_Performance.arch  
AGX5_FP16_Generic.arch  
AGX5_Small_NoSoftmax.arch
```

D2. Run area estimate

For Agilex 5:

```
dla_compiler --fanalyze-area --march $COREDLA_ROOT/example_architectures/AGX5_  
Generic.arch
```

Typical report contains:

```
ALMs  
ALUTs  
DSPs  
Registers  
M20Ks  
Memory ALMs
```

Part E - ResNet-50 example flow

E1. Download ResNet-50

```
cd $COREDLA_WORK  
omz_downloader --name resnet-50-tf --output_dir $COREDLA_WORK/demo/models/
```

E2. Convert to OpenVINO IR

```
omz_converter --name resnet-50-tf \  
--download_dir $COREDLA_WORK/demo/models/ \  
--output_dir $COREDLA_WORK/demo/models/
```

Verify:

```
ls $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32/
```

Expected:

```
resnet-50-tf.xml
resnet-50-tf.bin
```

E3. Compile for Agilex 5

```
cd $COREDLA_WORK/demo/models/public/resnet-50-tf/FP32

dla_compiler \
  --march $COREDLA_ROOT/example_architectures/AGX5_Generic.arch \
  --network-file ./resnet-50-tf.xml \
  --o $COREDLA_WORK/demo/RN50_AGX5_Generic_b1.aot \
  --fanalyze-performance
```

Success line:

```
Performance Estimate has finished for Neural Network 1
```

Check AOT file:

```
ls -lh $COREDLA_WORK/demo/RN50_AGX5_Generic_b1.aot
```

E4. View performance report

From the model FP32 directory:

```
cat compiled_model_dir/TensorFlow_Frontend_IR/reports/performance-report_0.txt
```

Key value:

```
FINAL THROUGHPUT
```

Part F - Software emulation flow

F1. Build runtime for emulation

```
cd $COREDLA_WORK/runtime
./build_runtime.sh --target_emulation
```

F2. Set variables

```
model_dir=$COREDLA_WORK/demo/models/public
image_dir=$COREDLA_WORK/demo/sample_images
cur_arch=$COREDLA_ROOT/example_architectures/AGX5_Generic.arch
```

F3. Run emulator inference

```
cd $COREDLA_WORK/runtime/build_Release/dla_benchmark

./dla_benchmark \
  -b 1 \
```

```

-niter 1 \
-nireq 1 \
-m $modeldir/resnet-50-tf/FP32/resnet-50-tf.xml \
-d HETERO:FPGA,CPU \
-i $imagedir \
-arch_file $curarch \
-dump_output \
-plugins emulation \
-bgr

```

Success line:

PASSED

Emulation is slow. It is for correctness/debugging, not performance.

Part G - Agilex 5 E-Series Modular Development Kit

G1. Your board

Board:

Agilex 5 FPGA and SoC E-Series Modular Development Kit
MK-A5E065BB32AES1

Relevant FPGA AI Suite design identifiers:

Design type	Identifier
OFS PCIe-attached	'agx5e_modular_ofs_pcie'
Hostless JTAG-attached	'agx5e_modular_jtag'
Hostless Spatial JTAG	'agx5e_modular_spatial_jtag'
On-chip parameter DDR-free	'agx5_modular_ddrfree'
DDR-overflow	'agx5_modular_ddrfree_ltdrwb'
SoC M2M	'agx5_soc_m2m'
SoC S2M	'agx5_soc_s2m'

G2. Recommended first board flow

Start with:

Hostless JTAG-attached
Identifier: agx5e_modular_jtag
Architecture: AGX5_Generic.arch

G3. Build JTAG runtime

```

cd $COREDLA_WORK/runtime
./build_runtime.sh -target_agx5_mdk_jtag_system_console

```

What this does: builds the FPGA AI Suite runtime plugin and dla_benchmark for the Agilex 5 Modular Development Kit Hostless JTAG System Console design.

Why it is required: the emulation runtime cannot talk to the real board. This target builds the runtime layer that uses Quartus System Console to access FPGA registers and DDR through JTAG.

G4. Build Agilex 5 JTAG bitstream


```
cd $COREDLA_WORK
$COREDLA_ROOT/bin/dla_build_example_design.py build \
  --licensed \
  --output-dir build_agx5_jtag_ed \
  --num-instances 1 \
  --seed 1 \
  agx5e_modular_jtag \
  $COREDLA_ROOT/example_architectures/AGX5_Generic.arch
```

What this does: creates the FPGA hardware design for the Agilex 5 Hostless JTAG example and runs Quartus to generate the programming bitstream.

Why it is required: the board must be configured with FPGA AI Suite IP that matches AGX5_Generic.arch. The runtime checks this architecture at load time.

The --licensed option forces licensed FPGA AI Suite IP generation. Use this when you have a valid FPGA AI Suite/CoreDLA license.

Expected output:

```
$COREDLA_WORK/build_agx5_jtag_ed/AGX5_Generic.sof
```

G5. Detect and program the board

Connect board power and USB-JTAG. Then detect the board:

```
jtagconfig
```

Expected example output:

```
1) Agilex_5E MDK Carrier [1-8.1]
   0364F0DD   A5E(C065BB32AR0|D065BB32AR0)
   020D10DD   VTAP10
```

What this does: verifies that Quartus JTAG tools can see the USB-JTAG cable and Agilex 5 device chain.

Why it is required: if jtagconfig cannot see the board, programming and hardware inference cannot work.

Set a stable JTAG clock:

```
jtagconfig --setparam 1 JtagClock 16M
```

What this does: lowers the JTAG cable clock to 16 MHz.

Why it is required: the JTAG design example documentation recommends 16 MHz or lower to reduce System Console/JTAG instability.

Program the FPGA:

```
cd $COREDLA_WORK/build_agx5_jtag_ed
quartus_pgm -c 1 -m jtag -o "p;AGX5_Generic.sof"
```

What this does: downloads the AGX5_Generic.sof bitstream into the Agilex 5 FPGA over JTAG.

Why it is required: the FPGA must contain the agx5e_modular_jtag hardware design before dla_benchmark can run on hardware.

Success looks like:

Configuration succeeded at device index 1
Quartus Prime Programmer was successful. 0 errors, 0 warnings

JTAG is slow. Use -nireq=1 for JTAG examples.

G6. Run ResNet-50 on real Agilex 5 hardware over JTAG

After programming AGX5_Generic.sof, set the runtime variables:

```
export PATH=/opt/altera/syscon/bin:/opt/altera/qcore/linux64:$PATH
export QUARTUS_ROOTDIR=/opt/altera/quartus
export DLA_SOF_PATH=$COREDLA_WORK/build_agx5_jtag_ed/AGX5_Generic.sof

MODEL=$COREDLA_WORK/demo/models/public/resnet-50-tf/FP32/resnet-50-tf.xml
IMGDIR=$COREDLA_WORK/demo/sample_images
ARCH=$COREDLA_ROOT/example_architectures/AGX5_Generic.arch
PLUGIN_XML=$COREDLA_WORK/runtime/build_Release/plugins.xml
```

What this does: sets paths to the Quartus System Console tools, the programmed bitstream, the OpenVINO model, input images, architecture file, and runtime plugin XML.

Why it is required: dla_benchmark needs all of these paths to load the correct model, match the hardware architecture, and communicate with the FPGA plugin.

If the runtime cannot auto-detect the JTAG master, list System Console master paths:

```
system-console --cli
```

At the System Console prompt:

```
get_service_paths master
exit
```

For the successful Agilex 5 JTAG run, the correct master path was:

```
/devices/A5E(C065BB32AR0|D065BB32AR0)@1#1-8.1#Agilex_5E MDK Carrier/(link)/JTAG/alt_sld_fab_0_alt_sld_fab_0_sldfabric.node_0/phy_0/jtag_master_0.master
```

Set it:

```
JTAG_PATH='/devices/A5E(C065BB32AR0|D065BB32AR0)@1#1-8.1#Agilex_5E MDK Carrier
/(link)/JTAG/alt_sld_fab_0_alt_sld_fab_0_sldfabric.node_0/phy_0/jtag_master_0.
master'
```

Run hardware inference:

```
cd $COREDLA_WORK/runtime/build_Release/dla_benchmark
rm -rf TensorFlow_Frontend_IR network_directories.txt csr_log.txt

./dla_benchmark \
  -b=1 \
  -m=$MODEL \
  -d=HETERO:FPGA,CPU \
  -i=$IMGDIR \
  -niter=2 \
  -plugins=$PLUGIN_XML \
  -arch_file=$ARCH \
  -api=async \
  -perf_est \
  -nireq=1 \
  -dump_output \
```

```
-report_lsu_counters \
-bgr \
-jtag-path="$JTAG_PATH"
```

What this does: runs ResNet-50 inference through OpenVINO HETERO mode, using the FPGA AI Suite plugin to execute supported layers on the FPGA and CPU fallback if needed.

Why it is required: this is the real hardware validation step. It proves the runtime can communicate with the programmed FPGA AI Suite IP and execute inference on the board.

Key options:

Option	Meaning
'-m'	OpenVINO model XML
'-d=HETERO:FPGA,CPU'	Prefer FPGA, fall back to CPU if needed
'-plugins'	Runtime plugin XML for the hardware build
'-arch_file'	Must match the architecture used to build the bitstream
'-nireq=1'	Required/recommended for JTAG design example
'-perf_est'	Prints FPGA AI Suite performance estimate alongside measured results
'-report_lsu_counters'	Dumps memory access counter information
'-jtag-path'	Explicit System Console master path for CSR/DDR access

Successful hardware run indicators:

```
Using licensed IP
Runtime arch check passed.
Runtime build version check passed.
IP throughput per instance: 31.1680 FPS
IP clock frequency measurement: 326.8177 MHz
```

Example measured result from the successful Agilex 5 run:

```
system throughput: 3.5635 FPS
IP throughput per instance: 31.1680 FPS
estimated IP throughput per instance: 30.6566 FPS
```

The system throughput is lower than IP throughput because JTAG transfer/control overhead is slow. The IP throughput is the accelerator throughput.

G7. Permanent licensed Docker startup

For a node-locked Altera license, start Docker with host networking, USB pass-through, workspace mount, license mount, and license environment variables.

Create or update this host-side script:

```
mkdir -p ~/bin
nano ~/bin/start-fpga-ai-suite.sh
```

Use this content, adjusting the license filename if needed:

```
#!/bin/bash
set -e

CONTAINER_NAME="fpga-ai-suite-2026"
IMAGE_NAME="alterafpga/fpgaaisuite:2026.1.1-quartus"
WORKSPACE="$HOME/fpga-ai-work"
LICENSE_FILE="$HOME/altera_pro/LR-178044_License.dat"

if [ ! -f "$LICENSE_FILE" ]; then
    echo "ERROR: License file not found: $LICENSE_FILE"
    exit 1
```

```

fi

mkdir -p "$WORKSPACE"

if docker ps -a --format '{{.Names}}' | grep -qx "$CONTAINER_NAME"; then
    echo "Starting existing FPGA AI Suite container..."
    docker start -ai "$CONTAINER_NAME"
else
    echo "Creating new licensed FPGA AI Suite container..."
    docker run -it \
        --name "$CONTAINER_NAME" \
        --network host \
        --privileged \
        -v /dev/bus/usb:/dev/bus/usb \
        -v "$WORKSPACE":/workspace \
        -v "$LICENSE_FILE":/licenses/altera_license.dat:ro \
        -e LM_LICENSE_FILE=/licenses/altera_license.dat \
        -e ALTERAD_LICENSE_FILE=/licenses/altera_license.dat \
        "$IMAGE_NAME"
fi

```

Make it executable:

```
chmod +x ~/bin/start-fpga-ai-suite.sh
```

Daily startup:

```
~/bin/start-fpga-ai-suite.sh
```

Inside Docker, verify the license variables and tools:

```

echo $LM_LICENSE_FILE
echo $ALTERAD_LICENSE_FILE
ls -lh /licenses/altera_license.dat
which quartus_pgm
which system-console
which lmutil

```

If system-console or lmutil is not in PATH, add:

```

echo 'export PATH=/opt/altera/syscon/bin:/opt/altera/qcore/linux64:$PATH' >> ~/.bashrc
echo 'export QUARTUS_ROOTDIR=/opt/altera/quartus' >> ~/.bashrc
source ~/.bashrc

```

Part H - Object detection notes

FPGA AI Suite includes ported OpenVINO demo applications:

```

classification_sample_async
object_detection_demo_yolov3_async
segmentation_demo

```

The object detection demo is documented under the PCIe design example section and uses YOLOv3:

```

yolo-v3-tf
yolo-v3-tiny-tf

```

The sample command in the handbook is shown for a PCIe/Agilex 7 example. For the Agilex 5

E-Series Modular Development Kit, the relevant full application-style path is closer to:

```
agx5e_modular_ofs_pcie
```

JTAG can run inference, but it is not ideal for live/video object detection because USB-JTAG is slow.

H1. Download and compile YOLOv3 for AGX5

```
cd $COREDLA_WORK
omz_downloader --name yolo-v3-tf --output_dir $COREDLA_WORK/demo/models/

omz_converter --name yolo-v3-tf \
  --download_dir $COREDLA_WORK/demo/models/ \
  --output_dir $COREDLA_WORK/demo/models/

cd $COREDLA_WORK/demo/models/public/yolo-v3-tf/FP32

dla_compiler \
  --march $COREDLA_ROOT/example_architectures/AGX5_Generic.arch \
  --network-file ./yolo-v3-tf.xml \
  --o $COREDLA_WORK/demo/YOLOv3_AGX5_Generic_b1.aot \
  --fanalyze-performance
```

Part I - Troubleshooting

I1. Docker container name conflict

Error:

```
container name is already in use
```

Fix:

```
docker start -ai fpga-ai-suite-2026
```

or recreate:

```
docker rm fpga-ai-suite-2026
```

I2. OpenVINO not found

If this fails on host Ubuntu:

```
python3 -c "import opencvino; print(opencvino.__version__)"
```

that is expected if OpenVINO is only installed inside Docker. Enter container first:

```
docker start -ai fpga-ai-suite-2026
```

I3. Workspace permission denied

Inside Docker:

```
sudo chown -R fpga_ai_suite:fpga_ai_suite /workspace
```

14. License issue: No valid license for CoreDLA found

Symptoms in build.log:

```
No valid license for CoreDLA found.
Building unlicensed version.
```

First confirm the license file is mounted inside Docker:

```
echo $LM_LICENSE_FILE
echo $ALTERAD_LICENSE_FILE
ls -lh /licenses/altera_license.dat
```

Find lmutil if needed:

```
find /opt/altera -name lmutil 2>/dev/null
```

In the Quartus Docker image used here, lmutil was located at:

```
/opt/altera/qcore/linux64/lmutil
```

Add it to PATH:

```
export PATH=/opt/altera/qcore/linux64:$PATH
```

Check the container-visible host ID:

```
lmutil lmhostid
```

Check the FPGA AI Suite/CoreDLA features from the license. The useful feature IDs observed for FPGA AI Suite were:

```
6AF8_018B
6AF7_018B
```

Run diagnostics:

```
lmutil lmdiag -c /licenses/altera_license.dat 6AF8_018B
lmutil lmdiag -c /licenses/altera_license.dat 6AF7_018B
```

Success looks like:

```
This is the correct node for this node-locked license
```

If you see:

```
Invalid host
```

then the license is locked to a different NIC/host ID. Generate or rehost the license using the real Linux host Ethernet MAC, not the Docker virtual MAC. Confirm host MACs on the Linux host with:

```
ip link
```

For this setup, the corrected license was generated for primary computer ID:

```
e89744bdaefd
```

For node-locked licenses in Docker, start the container with:

```
--network host
```

so FlexLM can see the host network identity.

For the Agilex 5 JTAG design build, force the licensed IP path with:

```
$COREDLA_ROOT/bin/dla_build_example_design.py build \
--licensed \
--output-dir build_agx5_jtag_ed \
--num-instances 1 \
--seed 1 \
agx5e_modular_jtag \
$COREDLA_ROOT/example_architectures/AGX5_Generic.arch
```

The build log should include:

```
Created licensed IP.
Building licensed version.
```

I5. No JTAG hardware available

Possible causes:

1. Board not powered.
2. USB-JTAG cable not connected.
3. Container not started with USB pass-through.
4. Host USB permissions issue.

Use container command with:

```
--privileged -v /dev/bus/usb:/dev/bus/usb
```

I6. Accuracy check fails but emulator says PASSED

If emulator prints:

```
PASSED
```

then emulation ran. Accuracy mismatch usually means preprocessing or ground-truth mismatch. Try first without `-groundtruth_loc`, then inspect output files.

Final checklist

A complete working environment should pass:

```
python3 -c "import openvino; print(openvino.__version__)"
echo $COREDLA_ROOT
dla_compiler --version
quartus_pgm --version
ls $COREDLA_ROOT/example_architectures | grep AGX5
dla_compiler --fanalyze-area --march $COREDLA_ROOT/example_architectures/AGX5_
Generic.arch
```

A successful model flow should produce:

```
resnet-50-tf.xml
```

```
resnet-50-tf.bin  
RN50_AGX5_Generic_b1.aot  
Performance Estimate has finished  
PASSED from emulator
```